

Module - Plateformes de Développement

Spring

Support de cours

Par :

Dr. Ing. Faouzi JAIDI

faouzi.jaidi@gmail.com

2021-2022

Plan

- 1. Introduction
- 2. Maven
- 3. Logs : Log4J
- 4. Tests: JUnit
- 5. Tests & Qualité : Sonar
- 6. IOC & ID
- 7. Spring Boot
- 8. Spring Data JPA
- 9. Spring MVC REST
- 10. JSF
- 11. AOP
- 12. Projet
- 13. Spring Security
- 14. Extensions



Spring

T. U. : JUnit

JUnit

Tests

□ Tests de Logiciels

○ *Intérêts*

- Évaluer les produits d'activités (exigences, conception, code, etc.).
- Vérifier et valider le logiciel.
- Construire la confiance dans le niveau de qualité de l'objet de test.
- Trouver des défauts et des défaillances.
- Prévenir des défauts.
- Réduire le niveau de risque d'une qualité logicielle inadéquate.
- Gagner du temps, etc.

Tests

○ Mise en Application

— Pratiques :

SOUVENT	• Affichage en console
PARFOIS	• Traces et journalisation
RAREMENT	• Utilisation d'un debugger

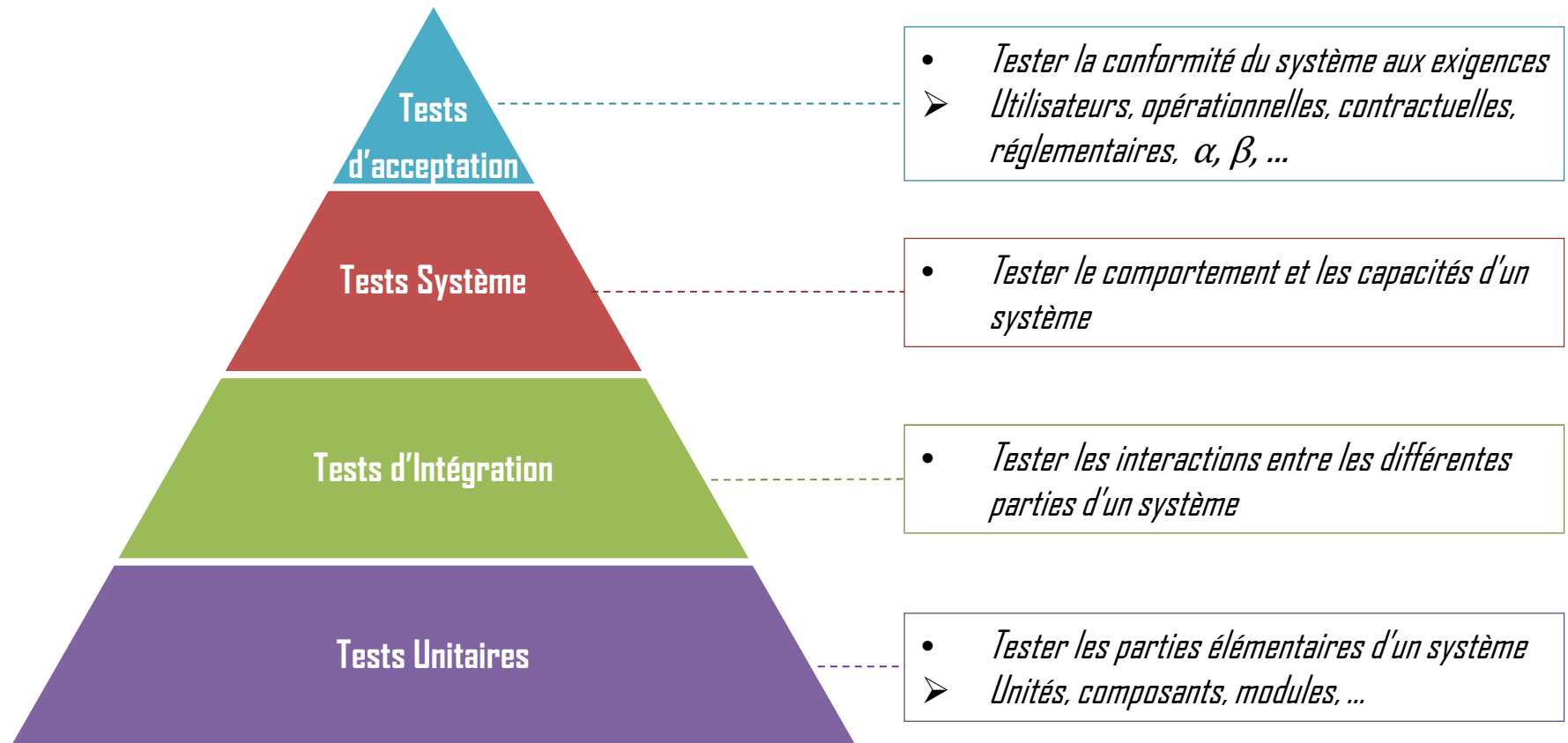
— Problèmes et limites :

- Difficulté à isoler les éléments,
- Perte de lisibilité (plusieurs affichages, etc.),
- Haut niveau (manque de recul, niveau d'indépendance faible, etc.)

➤ *Une stratégie / un processus de test est nécessaire.*

Tests

○ Niveaux de Tests



- Différents **types** et **techniques** de tests à appliquer dans chaque **niveau**.

[Plus de Détails : module Qualité, Tests et Validation de logiciels]

Tests Unitaires

□ Présentation

- Test de Composant $\equiv$ Test Unitaire $\equiv$ Test de Module.
- Rechercher des défauts et vérifier le bon fonctionnement des modules, objets, classes qui sont testables séparément.

○ Principe

- Un test unitaire repose sur deux principes simples:
 1. *Si ça fonctionne une fois, Alors ça fonctionnera toutes les autres fois.*
 2. *Si ça fonctionne pour quelques valeurs bien identifiées, Alors ça fonctionnera pour toutes les autres.*

Tests Unitaires

o Règles

- Quelques règles pour un test unitaire:
 - ❖ Doit être **isolé**: utilisation des bouchons (MOCK) «*Stub*», pilotes «*Driver*» ...
 - ❖ Doit être **déterministe**.
 - ❖ Doit être le **plus simple** possible.
 - ❖ Doit être lancé le **plus souvent** possible.
 - ❖ Doit être lancé le **plus tôt** possible.
 - ❖ Doit être lancé à **chaque** modification.
 - ❖ N'est pas de bout en bout.

Tests Unitaires

0 Frameworks

- Plusieurs frameworks de tests unitaires selon le langage de programmation:
 - Famille **xUnit**
 - PHPUnit [---> *PHP*]
 - SQLUnit [---> *SQL*]
 - JUnit [---> *JS*]
 - CUnit [---> *C*]
 - PyUnit [---> *Python*]
 - Junit [---> *Java*]
 - ...

JUnit

□ Présentation

- Un framework de T. U. pour le langage de programmation Java;
- Un projet «open source»;
www.junit.org
- JUnit est :
 - intégré par défaut dans les environnements de développement intégré (IDE) Java;
 - Facile et souple;
 - Basé sur des annotations (Java 5+) pour marquer les méthodes dédiées au test .

JUnit

□ Application

○ Packages

- `org.junit.Test`
- `org.junit.Assert`

○ Classes

- Nommage:

MaClasse.java  MaClasse**TEST**.java

- Pas de limite pour le nombre de tests au sein de la classe de test.

JUnit

o Méthodes

- Nommage: commence par test suivi du nom de la méthode

calculer()  testcalculer()

- Visibilité: *public*;
- Type de retour: *void*;
- Sans paramètres;
- Peut lever une exception;
- Annotée: `@Test`;
- Utilise des instructions de test (p.ex.: assertions);

JUnit

○ Assertions

- Liste : <http://junit.sourceforge.net/javadoc/>
- Exemples:
 - *fail()* / *fail(string)* : provoque l'échec de la méthode de test.
 - *assertTrue(boolean)* / *assertTrue(String, Boolean)* : fait échouer la méthode si le paramètre est faux.
 - *assertEquals(expected, actual)* / *assertEquals(String, expected, actual)* : fait échouer la méthode si les paramètres ne sont pas égaux [equals()].
 - *assertSame(expected, actual)* / *assertSame(String, expected, actual)* : fait échouer la méthode si les paramètres ne référence pas le même objet [==].
 - *assertNull(Object)* *assertNull(String, Object)* : fait échouer la méthode si le paramètre est différent de nul.
 - *assertNotNull, assertNotEquals, assertArrayEquals, ...*

JUnit

○ Annotations [Par ordre d'exécution]

- **@BeforeClass**: méthode exécutée avant le premier test, doit être static (méthode setUpBeforeClass() avec Junit 3.8).
- **@Before**: méthode exécutée avant chaque test (méthode setUp() avec Junit 3.8).
- **@Test**: méthode de test.

[@Test (timeout=...) : la méthode doit échouer si le temps d'exécution dépasse un seuil]

[@Test (expected=MonException .class) : vérifier qu'une méthode lève bien une exception]

- **@After**: méthode exécutée après chaque test (méthode tearDown() avec Junit 3.8).
- **@AfterClass**: méthode exécutée après le dernier test, doit être static (méthode tearDownAfterClass() avec Junit 3.8).



Spring

TP

